to a prompt. Here are some

sponse that follows a certain
asking the model to "Generate
uct" in generic sense, we ask
erate an ad copy for a
e AIDA framework –

e tone of a piece of text
mal, fun, serious & so on

a certain persona helps
onse. For ex.: "You are
product description for ..."
ext with a specific
others. This helps the
ce in b/w. For ex.,
f – – "

struggle in context aware
ask the model for
ones, it will duly

But if you were to ask it for the top qualities of the CO-1T headphone, it will not be able to provide an accurate response because it does not know about it.

In real app's, being able to add context to a prompt is key because this is what enables personalized generative AI for a team or company. It makes many use cases possible, such as intelligent assistants, customer support & productivity tools, that retrieve the right info from a wide range of sources & add it to the prompt.

Here's an example where we ask the model to list the features of the CO-1T wireless headphone w/o any additional context:

```
user_input = "What are the key features of CO-1T wireless headphone"
prompt = user_input
generate_text(prompt, temp = 0)
```

This generates a response that the model makes up since it doesn't have any info to refer it.

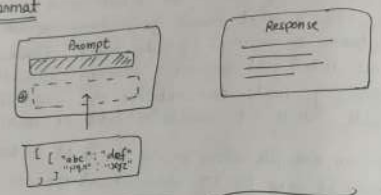
"The CO-1T wireless headphone is a high-quality – – "

And here's the same request to the models, this time with the product description of the product added as context.

```
context = """ Think back to the last time you were ...
                                                  """
user_input = "What are the key features – – "
prompt = f"""{context}
Given the information above, answer this question:
{user_input}"""
generate_text(prompt, temp=0)
```

Now, the model accurately lists the features of the model.
"The CO-1T wireless headphone have noise cancellation ..."

↳ Format



Adding output format requirements to a prompt

So far, we saw how to get the model to generate responses that follow certain styles or include specific info^n. But we can also get the model to generate responses in a certain format. Let's look at a couple of them: markdown tables & JSON strings.

Here, the task is to extract info^n from a list of invoices. Instead of providing the info^n in plain text, we can prompt the model to generate a table that contains all the info^n required.

prompt = """ Turn the following info^n into a table with columns Invoice Number, Merchant Name, and Account Number.

Bank Invoice: INVOICE #0521 MERCHANT ALLBIRDS ACC XXXXXX
" " " #6781 " SHOPPERS " XXXXX
" " " #0777 " CN TOWER " XXXXX
"""

generate_text (prompt, temp=0)

The response will

Here is a table form
Merchant Name, and

| Invoice Number | M
| --- |
| INVOICE #0521
| " #678
| " #077

Let me know i

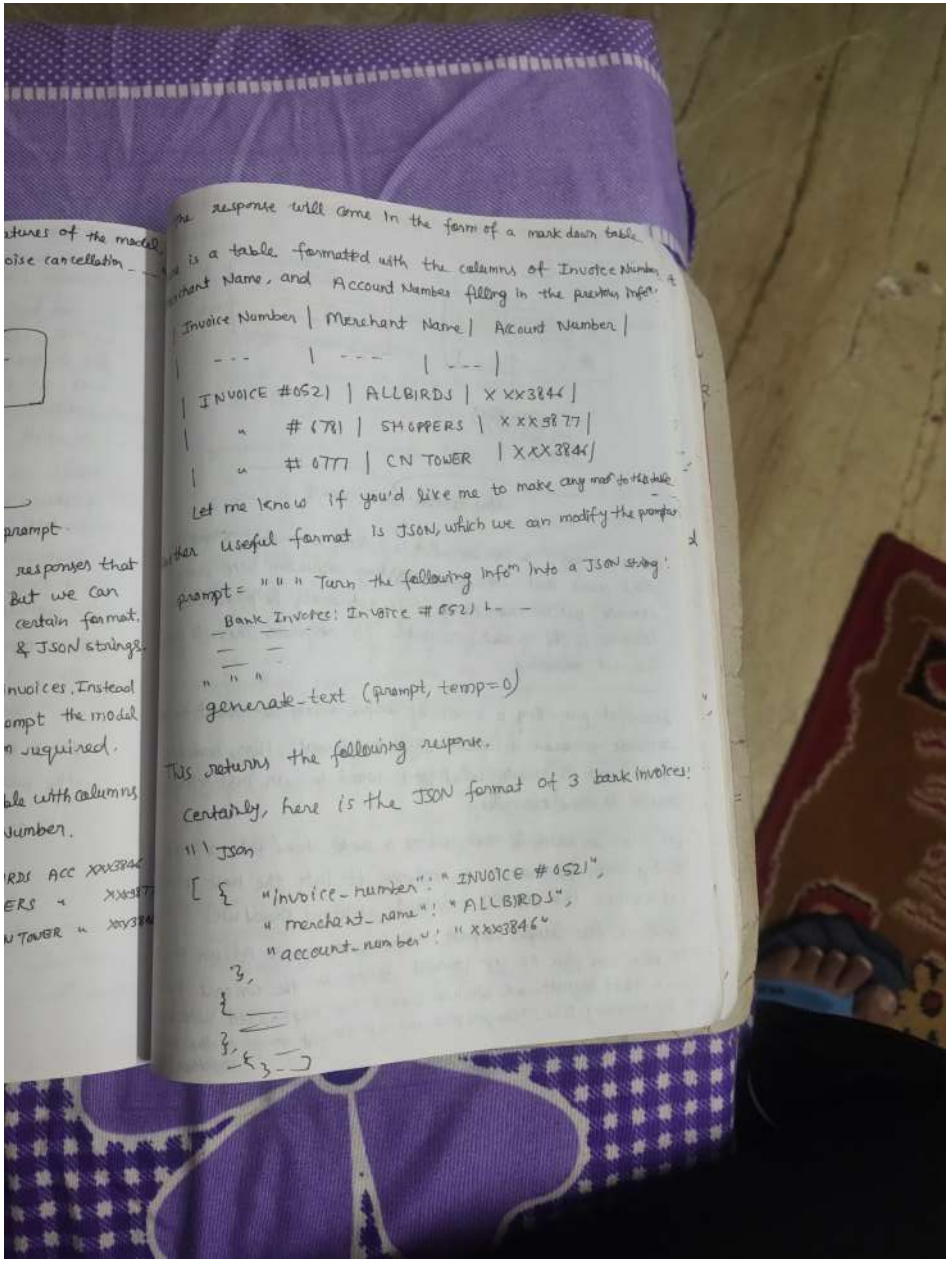
Another useful form

prompt = """ Tu
Bank Invoice:
"""

generate_tex

This returns the f

Certainly, here
```JSON
[ { "invoice
" merch
"accoun
},
{ ...
},
{ ... } ]
```



tures of the model
oise cancellation

prompt

responses that
But we can
certain format.
& JSON strings

invoices. Instead
ompt the model
n required.

ble with columns
Number.

RDS Acc XXX3846
ERS " XXX5877
N TOWER " XXX3846

The response will come in the form of a mark down table
is a table formatted with the columns of Invoice Number &
chant Name, and Account Number filling in the previous info:

Invoice Number	Merchant Name	Account Number
---	---	---
INVOICE #0521	ALLBIRDS	XXX3846
" #0781	SHOPPERS	XXX5877
" #0777	CN TOWER	XXX3846

Let me know if you'd like me to make any mod to the table

ther useful format is JSON, which we can modify the prompt

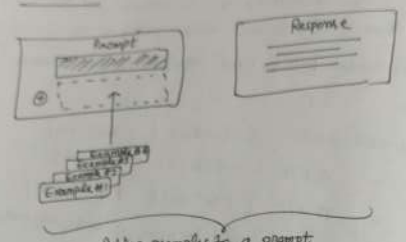
```
prompt = """ Turn the following info into a JSON string:
    Bank Invoices: Invoice #0521 ...
    ...
    """
generate_text (prompt, temp=0)
```

This returns the following response.

Certainly, here is the JSON format of 3 bank invoices:

```
"""JSON
[ {  "invoice_number": "INVOICE #0521",
     "merchant_name": "ALLBIRDS",
     "account_number": "XXX3846"
  },
  { ...
  },
  { ... }
```

Examples



Adding examples to a prompt

All our prompts so far use what is called zero-shot prompting, which means that we are providing instruction w/o any example. But in many cases, it is extremely helpful to provide examples to the model to guide its response. This is called few-shot prompting.

Few-shot prompting is especially useful when we want the model response to follow a certain style or format. Also, sometimes it is hard to explain what you want in an instruction & easier to show examples.

Let's use an example task, where a model should take a request coming from a human & rephrase it into the most accurate utterance than an AI virtual assistant should use.

We'll use this example request: "Send a message to Alison to ask if she can pick me up tonight to go to the concert together". Given that request, we should expect the rephraised utterance to be something like: "Can you pick me up tonight to go to the concert together?"

First, let's generate a example.

Here's the prompt:

prompt = """ Turn
-into the correct
Send a message
up tonight to

generate_text (

The response we
follow the style
one-line rephrasi
Instead, it gener

"OK, I will send
she can pick y
together. Please
inquiry so I

Now let's tratify
examples of how

user_input = "send a
pick m

prompt = f """ Tu
assistant into th
Message: Ask me a
me October
Action: Can you

First, let's generate a response w/o giving the model an example.

Here's the prompt:

```
prompt= """ Turn the following message to a virtual assistant
 into the correct action:
    Send a message to Alison to ask if she can pick me
    up tonight to go to the concert together" " "

generate_text (prompt, temp=0)
```

The response we get is not wrong, but it doesn't follow the style that we need, which is a simple one-line rephrasing of the original request.

Instead, it generates an email:

"OK, I will send a message to Alison asking if she can pick you up tonight to go to the concert together. Please provide me with details of your inquiry so I can assist you better!"

Now let's ratify the prompt by adding a few examples of how we expect the output to be.

```
user_input = "send a message to Alison to ask if she can
              pick me up tonight to go to concert together"

prompt = f""" Turn the following message to a virtual
 assistant into the current action:
 Message: Ask me aunt if she can go to the JDRF walk with
          me October 6th
 Action: can you go to the Jdrf walk with me October 6th
```

Message: Ask Eliza what should I bring to the wedding tomorrow

Action: ~~I am s~~: what should I bring to the wedding tomorrow.

Message: Send message to supervisor that I am sick and will not be in today

Action: I am sick & will not be in today

Message: {user_input}" " "

generate_text (prompt, temp=0)

This time, the style of the response is exactly how we want it.

"can you pick me up tonight to go to to the concert together?"

↳ Chain of Thought

One specific way to provide examples in a prompt is to show responses that include a reasoning step. This way, we are asking the model to "think" first rather than going straight to the response. In tasks involving mathematical questions, for example, there is a huge difference b/w directly giving the answer & adding a response step in between.

This concept is called "chain of thought" prompting, introduced by Wei et al. Let's look at an example from paper which illustrates this idea.

First, let's look at a prompt
It contains one example of
answer, w/o any intermed
contains the new question

prompt = f" " " Q: Ro
2 more cans of tennis
balls. How many to

A: The answer is

Q: The cafeteria
20 to make l
apples do they

A: The answ

Q: A box hav
are red
in the

A: " " "

generate_text (

We get the follow
answer (12). An
the answer firs

The answer is
are red and
adding 4 re
& 6

First, let's look at a prompt w/o a chain of thoughts. It contains one example of a question followed by the answer, w/o any intermediate calculation step. It also contains the new question we want to answer.

```
prompt = f """ Q: Roger has 5 tennis balls. He buys
    2 more cans of tennis balls. Each can has 3 tennis
    balls. How many tennis balls does he have now?
    A: The answer is 11.
    - - -
    Q: The cafeteria had 23 apples. If they used
    20 to make lunch & bought 6 more, how many
    apples do they have?
    A: The answer is 9.
    - - -
    Q: A box has 10 balls & a half of the balls
       are red balls. How many red balls are
       in the box if 4 red balls are added?
    A: """
generate_text(prompt, temp=0)
```

We get the following response, which is an incorrect answer (12). And notice that the response gives the answer first, similar to the example shown.

"The answer is 12. There are 5 balls that are red and 5 balls that are not red and adding 4 red balls would make it 9 red & 6 not red."

Now, let's repeat that, this time with a chain of thought. Now, the example answer contains a reasoning step, describing the calculation logic to get to the final answer, before giving the final answer.

prompt = f""" Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5+6=11.
The answer is 11

Q: The cafeteria had 23 apples. If they used 20 to make lunch & bough 6 more, how many apples do they have?

A: The cafeteria started with 23 apples. They used 20 to make lunch, so they have 23-20=3 apples. They bought 6 more apples, so they have 3+6=9 apples. The answer is 9.

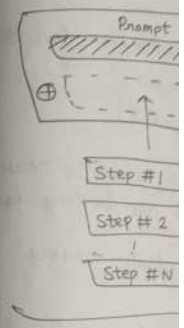
Q: A box has 12 balls & a half of the balls are red balls. How many red balls are in the box if 4 red balls are added?

A: """

generate_text(prompt, temp=0)

And we get the corre
response following the
"There are 12/2 =
red balls are a
6+4 = 10. The a

Steps

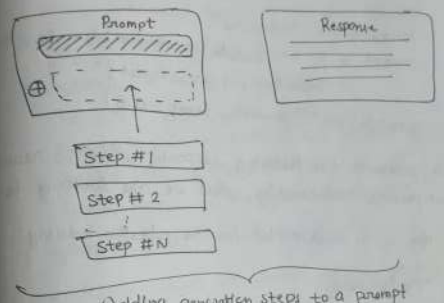


To steer the mod
responses, it can
model to genera
the final outpu
steps helps
it generates t

we get the correct answer this time (10), with the response following the style of the example given.

"There are 12/2 = 6 red balls in the box. When 4 more red balls are added, the no. of red balls becomes 6+4 = 10. The answer is 10."

steps



Adding generation steps to a prompt

To steer the model toward generating higher-quality responses, it can be helpful to add instructions for the model to generate intermediate steps before generating the final output. The info generated during these steps helps enrich the model's context before it generates the final response.

There could be another scenario where we specifically need the response to contain specific inform. In this case, we can specify what we need in the prompt.

→ Let's use an example of generating startup ideas. We can get the model to directly generate an idea for a given industry, like so:

```
user_input = "education"
prompt = f"""Generate a startup idea for this
            industry: {user_input}""""
generate_text(prompt, temp=0.5)
```

This generates the following response, which is reasonable, but perhaps not exactly what we are looking for.

Here is a startup idea for the education industry:

"MathsMadeEasy"

Description:

"MathsMadeEasy" is an innovative K-12 ed-tech startup that aims to revolutionize the way young students learn mathematics with a vision to eliminate math-related anxiety & foster a

- - - -

Approach & Features:

1. Gamified Learning: . - - . :
2. Personalized " : - —

Alternatively, we can ask the model to generate inform in steps, such as describing the probm to be solved & the target audience experiencing this probm.

user_input = "educat

prompt = f"""Gener

{user_input}

First, describe th

Next, describe t

Next, describe

for the target

Next, provide a

Use the follow

Industry: <t

The Probm: <

Audience: <

Startup Idea:

Startup Name:

generate_text

This provides a richer

Industry: Educa

The Probm: Man

Audience: Re

Startup Idea: O

Startup Name:

Our platform

→ Conclusion . A prompt

instructn, though the

the level of quality

Each building block a

level to enhance

we specifically need
In this case, we can
t.
g startup ideas. We
nerate an idea

dea for this
" "

h is reasonable
oking for.

ndustry:

h startup that
nts learn mathematics
ety & foster a

fom in steps,
he target

```
user_input = "education"
prompt= f """ Generate a startup idea for this industry:
        {user_input}
```

First, describe the problem to be solved.
Next, describe the target audience of this startup idea.
Next, describe the startup idea & how it solves the problem for the target audience.
Next, provide a name for the given startup.

Use the following format:

Industry : <the given industry>
The Probm : <the given problem>
Audience : <the given target audience>
Startup Idea : <the given idea>
Startup Name : <the given name>" " "

```
generate_text(prompt, temp=0.9)
```

This provides a richer description of the startup idea.

Industry: Education
The Probm: Many people today struggle - - -
Audience: Recent - - -
Startup Idea: Online community - -
Startup Name: "Nurture Nest"
Our platform offers a - - -

Conclusion . A prompt can be as simple as a single line of instructn, though the more specific the prompt is, the higher the level of quality & accuracy you can expect from response. Each building block added a prompt provides a diff. type of level to enhance the quality of the response.